Product Requirements Document

Outcome-driven networking for conferences

Cloud Native Hackathon, Salt Lake City · Build day: Monday 9 November 2026 · Version 1.0, 4 November 2026 · Status: Draft for team review

---

1. Summary

Conference attendees arrive with specific questions and outcomes, such as how a piece of technology works internally or how to get a job at a particular organisation, but have no way to find the few people in the building who can help. This product lets a user state that question or outcome, finds attendees with evidence they can resolve it, and arranges a short, focused conversation with minimal interaction.

What makes it different: existing conference matchmaking platforms (for example Brella, Grip and Swapcard) match people on broad interests. Conferences leave little time to get to know people broadly. We match on a specific ask, backed by evidence from each person's professional background, and measure success by outcomes achieved.

For the hackathon we build one end-to-end flow on Elasticsearch, instrumented with Dash0, shipped through Sonar/Gitar quality gates, and pitched for the Merge Forward accessibility prize.

2. Problem

Networking is the most underused part of a conference. Today people find contacts by reading lanyards for a home institution or country and hoping. Most valuable conversations never happen because:

- Attendees cannot tell who in the room has the specific knowledge, role or connections they need.
- There is no time for broad get-to-know-you conversations; a focused 10-minute question is realistic, an open-ended chat is not.
- Busy experts are flooded with vague requests and ignore them, so the people best able to help are the hardest to reach.
- Scheduling around talks is manual and drains attention away from the conference itself.

3. Goals, non-goals and anti-goals

Goals

- A user can go from a vague need to a specific, answerable ask and a booked conversation in under 5 minutes.
- Every suggested person comes with visible evidence of why they can help.
- Receivers get fewer, better requests that state the question up front.
- Engagement comes from outcomes: questions answered, introductions made, progress toward a goal.
- Works on laptop, tablet and phone.

Non-goals (for the hackathon)

- Research and education conference connectors (ORCID, OpenAlex, PubMed, ERIC) beyond one pre-loaded profile.
- Sponsor matching, auto-moving meetings and venue logistics.
- Native mobile apps; a responsive installable web app covers all devices.

Anti-goals (failure even if it technically works)

- Maximising time-in-app. The spec calls for minimising distraction during the conference.
- Leaderboards for swipes or meetings, which reward spamming requests.
- Scraping LinkedIn, ResearchGate or any source against its terms of use.

- Generic interest matching that looks like every competitor.

4. Users

User	What they want	What they fear
Requester (knowledge)	Understand a specific mechanism, e.g. how Kubernetes scheduler preemption chooses what to evict	Wasting a slot on someone who only knows the basics
Requester (career)	Get a job at a named organisation or break into a field	Cold approaches that feel awkward or go nowhere
Receiver (expert)	Help people with questions they can actually answer, on their own terms	A flood of vague requests and lost conference time
Sponsor (later phase)	Meet attendees who fit their product, at preferred times such as lunch	Booth traffic that never converts

5. Core user flow

The one flow the demo must show working end to end:

1. **Sign in (under 2 minutes).** The user signs in with GitHub and answers fixed questions: country, workplace and hiring status. These act as hard filters. The app shows “Here’s what we learned about you”, a profile card built from their repos, languages and bio, which they edit rather than write.
2. **State an ask.** The user types a question or outcome, e.g. “I want to understand how preemption works in the Kubernetes scheduler” or “I want a job at Grafana Labs”.
3. **Refine.** AI turns a vague ask into a specific one that can be answered in a 10-minute conversation, classifies its type (knowledge, career, collaboration, hiring) and suggests a meeting length.
4. **See evidence-backed people.** A ranked list of attendees, each card showing highlighted proof (“Maintains pkg/scheduler/preemption”), their hard-filter badges (“not hiring”) and whether they are also looking for someone like the requester. A search box narrows the list.
5. **Send or reject.** The user swipes right to send the refined ask, or left to reject. Batch rejection and a soft blocker apply (section 6).
6. **Receiver responds.** The receiver sees the question up front and can accept, decline or answer in a short reply without a meeting.
7. **Schedule.** On accept, the app proposes a free slot from both schedules, never less than 1 hour away.
8. **After.** The user marks the ask resolved; the app suggests connecting on social platforms.

6. Functional requirements

Priority key: P0 = built and demoed on 9 November. P1 = stretch goal on the day. P2 = designed and shown in the pitch, not built.

ID	Requirement	Priority
FR-1	GitHub sign-in builds a profile from repos, languages, bio and organisations	P0
FR-2	Fixed onboarding questions (country, workplace normalised to a ROR ID, hiring status) act as hard filters	P0
FR-3	“Here’s what we learned about you” editable profile card	P0
FR-4	Ask box accepts a free-text question or outcome	P0
FR-5	AI refinement makes the ask specific, classifies its type and suggests a meeting length	P0

ID	Requirement	Priority
FR-6	Evidence-backed ranked results with highlighted proof and hard-filter badges on each card	P0
FR-7	Mutual-interest check ranks people higher when their own asks match the requester	P0
FR-8	Swipe accept/reject; batch rejection of lower-ranked results after a set number of rejections for an intent	P0
FR-9	Soft blocker: people rejected for an intent are hidden both ways while that intent is on; the user can switch it off	P0
FR-10	Request carries the refined question; receiver can accept, decline or answer in a short reply	P0
FR-11	Slot suggestion from both schedules with a 1-hour lockout	P0
FR-12	“N people are looking for someone like you” counter	P0
FR-13	Responsive layout for phone, tablet and laptop	P0
FR-14	Request scoring and ranking in the receiver’s queue; auto-decline rules by time, type and a daily cap	P1
FR-15	Conversation helper corrects wording and tone; it does not suggest an opening message	P1
FR-16	Live conference pulse: trending topics and asks in the room	P1
FR-17	Profile and request analytics for each user	P1
FR-18	Outcome tracker, e.g. “Job at Grafana: 2 people met, 1 referral”	P2
FR-19	Session-break match drops (“4 people from the talk you just left”)	P2
FR-20	Post-meeting kudos feeding profile ratings; shareable recap card; social connect suggestions	P2
FR-21	Sponsor profiles (product, offer, booth location) and attendee groups for preferred slots such as lunch	P2
FR-22	ORCID sign-in with OpenAlex, PubMed and ERIC enrichment; CV and LinkedIn data-export upload	P2
FR-23	Auto-moving meetings and nearest-location logistics	P2

7. Non-functional requirements

Area	Requirement
Performance	Match results return in under 2 seconds for 95% of requests (the Dash0 SLO)
Freshness	A new sign-up is searchable within about 1 second (Elasticsearch default refresh)
Devices	Usable on phone, tablet and laptop from one codebase
Accessibility	Keyboard navigable, screen-reader labels, WCAG AA colour contrast
Privacy	Only public data and data the user uploads; per-source consent at sign-in; no scraping
Honesty	Synthetic demo profiles are labelled as synthetic in the UI
Quality	Every change merges through the gates in section 11

8. Technical approach

Stack

Layer	Choice	Reason
App	Next.js, TypeScript, Tailwind; installable web app	One codebase and one language for every device
Sign-in	GitHub OAuth through Auth.js	One click; profile data arrives with it
Search and matching	Elasticsearch (Elastic Cloud trial)	Core data layer for all searchable data; avoids running it on a laptop alongside everything else
Transactional data	Postgres	Accounts, requests, meetings and accept/decline state
AI	OpenAI (paid with the $50 per-attendee credits)	Ask refinement, explanations, wording help
Embeddings	semantic_text field with an inference endpoint inside Elasticsearch	One embedding model for every document
Observability	OpenTelemetry auto-instrumentation exporting to Dash0	Traces, metrics, logs and real-user monitoring
Code quality	GitHub, Gitar, SonarQube Cloud, GitHub Actions	Section 11

Request pipeline

Every ask runs through four steps, each traced as its own span in Dash0:

1. **Understand the request.** An AI call refines the ask and classifies its type. The result is cached by a hash of the ask.
2. **Search Elasticsearch.** A hybrid search over evidence documents, filtered and grouped by person (section 9).
3. **Check mutual interests.** The percolator finds which candidates are themselves looking for someone like the requester.
4. **Generate explanations.** An AI call writes one “why this person” line per result from the highlighted evidence. Results are cached by person and ask.

Data split

All searchable data lives in Elasticsearch: the 500 synthetic demo profiles and every participant who joins on the day share the same indices. Postgres holds transactional records only. Putting day-of joiners in a separate database would bypass Elasticsearch in the live demo and weaken the core-data-layer claim.

9. Elasticsearch design

Index	Contents	Used for
people	Profile, hard-filter fields (country, ROR workplace ID, hiring), tags, availability, semantic_text summary	Filters, badges, slot overlap
evidence	One document per commit area, repo, paper, talk or abstract, with person_id and semantic_text	Finding proof a person can resolve an ask
asks	Each user’s active asks stored as percolator queries	Mutual-interest check and “people looking for you”

Capabilities we showcase (the “top layer”):

- **Hybrid search:** keyword (BM25) and semantic retrieval merged with reciprocal rank fusion.
- **Filters:** hard blockers in a bool filter; soft blockers as a toggleable must_not on rejected people.
- **Collapse by person:** evidence hits grouped into one result per attendee.
- **Highlighting:** the exact evidence behind each match, shown on the card.
- **Percolator:** reverse search, so a new joiner instantly surfaces to people already looking for them.
- **Boosting:** free-slot overlap and profile rating raise a result's rank.
- **Aggregations:** live conference pulse and per-user analytics.
- **Near-real-time refresh:** a judge who signs in on stage appears in others' results within about a second.

Demo data

500 synthetic attendees with realistic evidence documents, generated with an LLM and loaded through the same ingestion path as real users, plus one ORCID-based researcher profile to show the pipeline extends to research conferences. Live participants join the same indices on the day.

10. Data sources

Source	Conference type	What it gives	Phase
GitHub	Tech	Identity anchor, repos, languages, bio, organisations	Hackathon
ROR	All	One ID per institution, so workplace and country filters match	Hackathon
Fixed questions	All	Hard filters and intent	Hackathon
Conference program	All	Abstracts submitted to this event	Hackathon (seeded)
Stack Exchange API	Tech	Tags someone answers on	Later
DBLP, arXiv	Tech	Computer science papers and preprints	Later
ORCID	Research, education	Identity anchor, employment, works	Later (one demo profile)
OpenAlex	Research, education	Abstracts, topics, co-authors by DOI	Later (one demo profile)
Semantic Scholar	Research	Papers, citations, SPECTER2 embeddings	Later
PubMed / MeSH	Medical	Standard specialty tags	Later
ERIC	Education	Education research	Later
CV, LinkedIn data export	All	Background for people who don't publish	Later

LinkedIn and ResearchGate have no public API for this use; their data only enters through the user's own export. Access limits come from the team's research and general knowledge and should be checked before 9 November.

11. Engineering quality (Sonar track)

Every feature lands as a pull request. Merging is blocked until all gates pass, enforced by branch protection:

1. Lint and format

2. Type-check
3. Unit and integration tests with a coverage threshold
4. SonarQube Cloud quality gate: coverage, duplication, code smells
5. Gitar review against project rules in .gitar/review/ (conventions, tests required, caching, no repeated calls in loops), with CI failure diagnosis and auto-apply enabled

Caching: AI refinements and explanations cached by input hash; Elasticsearch request cache for aggregations; cache hit rate exported as a metric.

12. Observability (Dash0 track)

- **Traces:** one span per pipeline step (understand, search, mutual check, explain), so the slowest step is visible at a glance.
- **Metrics:** response time per step (median and 95th percentile), AI tokens and cost per ask, cache hit rate.
- **Real-user monitoring:** the engagement funnel from sign-in to first ask, first request sent and first ask resolved.
- **SLO and dashboard:** match results under 2 seconds for 95% of requests, on a Dash0 dashboard with an alert.
- **Demo story:** use Agent0 to find a real slow step during the day (for example, uncached explanations), fix it and show the before and after.

13. Engagement

Engagement means features that make people want to use the platform. Each one is tied to an outcome, not screen time.

Moment	Feature	Priority
First 2 minutes	“Here’s what we learned about you” card; profile strength meter	P0 / P1
Curiosity	“N people are looking for someone like you”	P0
Payoff	Mutual-match and “your question was answered” moments	P0
Trust	Specific evidence on every card instead of “similar interests”	P0
Lower fear	Conversation helper for wording and tone	P1
Alive during the event	Conference pulse; session-break match drops	P1 / P2
Progress	Outcome tracker toward the user’s goal	P2
Reward and memory	Kudos, sponsor perks, shareable recap card, social connects	P2

14. Hackathon tracks

Track	Prize	What judges look for	How we meet it
Elasticsearch	$4,000	Elasticsearch as the core data layer; smart keyword, vector or hybrid search	All searchable data and matching in Elasticsearch; section 9
Sonar (Gitar)	$4,000	Features through Gitar-reviewed PRs; project rules; CI diagnosis and auto-apply	Section 11
Dash0	$4,000	Instrumentation depth; Agent0 or MCP fixing something real; dashboard or SLO	Section 12

Track	Prize	What judges look for	How we meet it
Merge Forward	iPad	More inclusive and accessible conferences	Any device; wording help; access to experts that doesn't depend on who you know
Best overall	NVIDIA DGX Spark	How well it makes conferences easier	Judges sign in and get their own matches live
Radar	$4,000	Kubernetes, GitOps, AI SRE	Not entered: too heavy for the time available

A project only counts for a sponsor track if it genuinely uses that sponsor's tool.

15. Success metrics

Product

- Share of asks resolved (answered in chat or a meeting held)
- Time from sign-in to first request sent (target: under 5 minutes)
- Request acceptance rate, and share answered without a meeting
- Receivers' rating of request relevance

Hackathon

- The core flow works live with a judge's own GitHub account
- Each entered track's judging criteria visibly met in the demo
- Match latency SLO shown green on the Dash0 dashboard

16. Plan

Before 9 November

- Create Gitar accounts, install the GitHub App on the repo and write the review rules
- Set up SonarQube Cloud and the GitHub Actions pipeline with branch protection
- Create the Dash0 trial and confirm OpenTelemetry export from a hello-world app
- Start the Elastic Cloud trial and confirm semantic_text, reciprocal rank fusion and percolator work on it
- Create the GitHub OAuth app
- Generate the 500 synthetic profiles and a sample conference schedule
- Ask the organisers whether code may be written before the day

Build day (Mountain Time)

Time	Work
11:00 to 1:00	Indices, ingestion, sign-in, and the four-step pipeline working through the API
1:00 to 2:30	Lunch and the Kelsey Hightower panel
2:30 to 4:30	Ask box, result cards, swipe, receiver response, slot suggestion
4:30 to 5:30	Dash0 dashboard and Agent0 fix, demo data check, rehearsal
5:30	Submissions close; presentations and judging follow

17. Risks and confidence

Item	Confidence	Note
Elasticsearch is the right core and strongest track fit	High (~85%)	The product is a search problem
Sonar/Gitar track is worth entering	High (~80%)	Only if set up before the day
Dash0 is a better third track than Radar	Medium-high (~70%)	Agreed by the team
Demoing at a tech conference beats a research one	Medium-high (~70%)	Judges have GitHub, not ORCID
The P0 scope fits in 5 to 6 hours	Medium (~55%)	Assumes 2 to 3 people and prep done; low if solo
The stack suits the team	Medium (~60%)	Depends on skills not yet confirmed
Elastic trial covers every showcased feature	Unverified	Check before the day

18. Open questions

- How many people are on the team, and who owns which part?
- Which languages and tools is the team strongest in? This may change the stack.
- What is the product called?
- May code be written before 9 November?
- Does the demo use a seeded conference schedule, or can attendees enter availability?

Sources

- Event page: https://luma.com/cnh?tk=SAKbX4
- Hackathon details: https://wemakedevs.org/hackathons/cnh
- Resources: https://wemakedevs.org/hackathons/cnh/resources